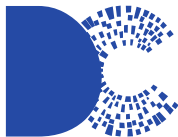




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Programação paralela (OpenCL)

Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

Introdução

- ▶ Modelo paralelo de programação (vetorial)
 - ▶ O OpenCL é um padrão aberto de programação que suporta plataformas heterogêneas de hardware, como AMD, Intel e Nvidia

Introdução

- ▶ Modelo paralelo de programação (vetorial)
 - ▶ O OpenCL é um padrão aberto de programação que suporta plataformas heterogêneas de hardware, como AMD, Intel e Nvidia
 - ▶ Múltiplos núcleos de processamento (CPU)

Introdução

- ▶ Modelo paralelo de programação (vetorial)
 - ▶ O OpenCL é um padrão aberto de programação que suporta plataformas heterogêneas de hardware, como AMD, Intel e Nvidia
 - ▶ Múltiplos núcleos de processamento (CPU)
 - ▶ Processadores gráficos de propósito geral (GPU)

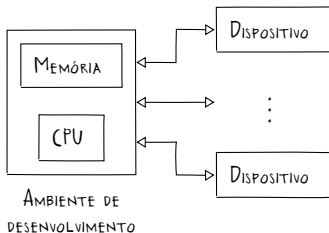
Introdução

- ▶ Modelo paralelo de programação (vetorial)
 - ▶ O OpenCL é um padrão aberto de programação que suporta plataformas heterogêneas de hardware, como AMD, Intel e Nvidia
 - ▶ Múltiplos núcleos de processamento (CPU)
 - ▶ Processadores gráficos de propósito geral (GPU)
 - ▶ Hardware reprogramável (FPGA)

Introdução

► *Host* × *Device*

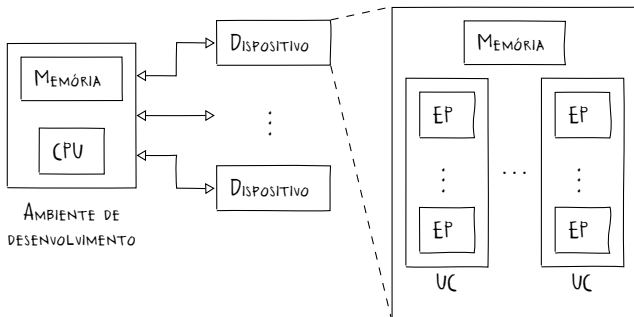
- O ambiente de desenvolvimento (*host*) é responsável pelo gerenciamento das tarefas e transferência de dados com pelo menos um dispositivo (*device*)



Introdução

► *Host* × *Device*

- O ambiente de desenvolvimento (*host*) é responsável pelo gerenciamento das tarefas e transferência de dados com pelo menos um dispositivo (*device*)



Todo o processamento é realizado no dispositivo, através de suas unidades de computação (UC) e de seus respectivos elementos de processamento (EP)

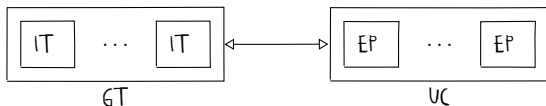
Introdução

- ▶ Função de núcleo (*kernel*)
 - ▶ É o código que será executado no dispositivo, sendo mapeado como item de trabalho (IT) em cada elemento de processamento (EP) e grupo de trabalho (GT) nas unidades de computação (UC)



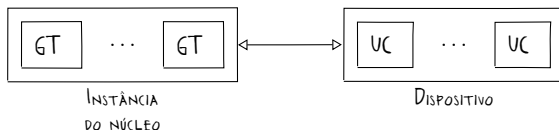
Introdução

- ▶ Função de núcleo (*kernel*)
 - ▶ É o código que será executado no dispositivo, sendo mapeado como item de trabalho (IT) em cada elemento de processamento (EP) e grupo de trabalho (GT) nas unidades de computação (UC)



Introdução

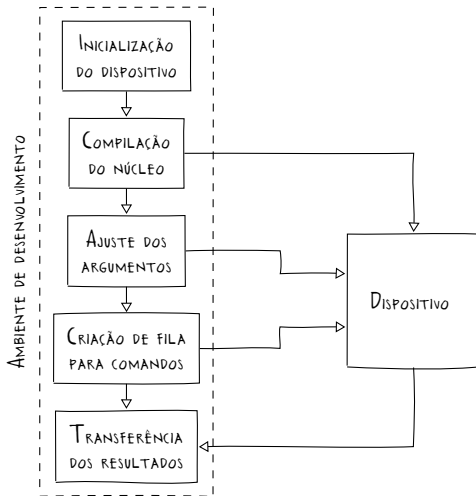
- ▶ Função de núcleo (*kernel*)
 - ▶ É o código que será executado no dispositivo, sendo mapeado como item de trabalho (IT) em cada elemento de processamento (EP) e grupo de trabalho (GT) nas unidades de computação (UC)



Cada função do núcleo é alocada em um conjunto de grupos de trabalho do dispositivo

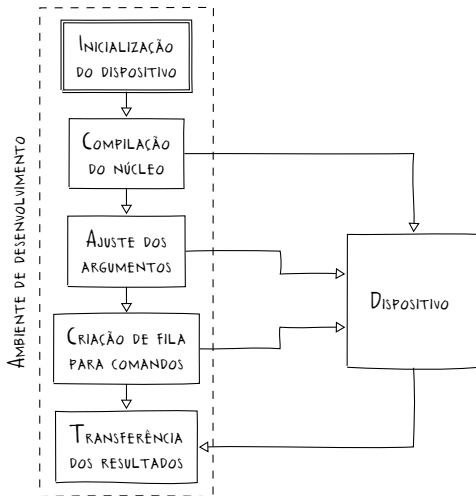
Introdução

► Fluxo de desenvolvimento



Introdução

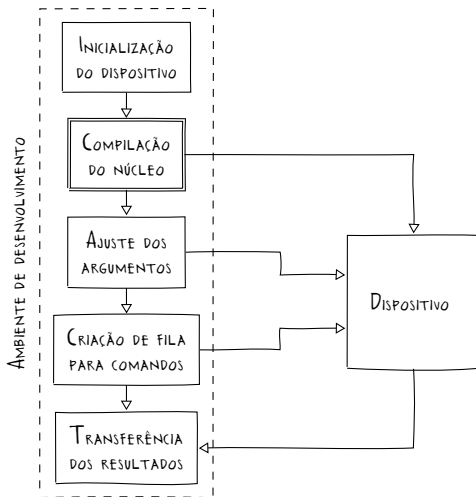
► Fluxo de desenvolvimento



Busca e inicialização de dispositivos nas plataformas

Introdução

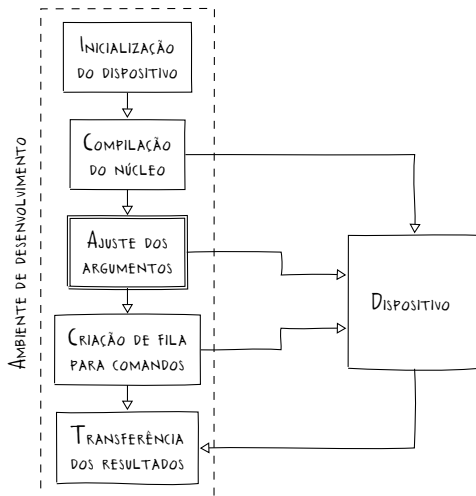
► Fluxo de desenvolvimento



Compilação da função do núcleo para o dispositivo alvo

Introdução

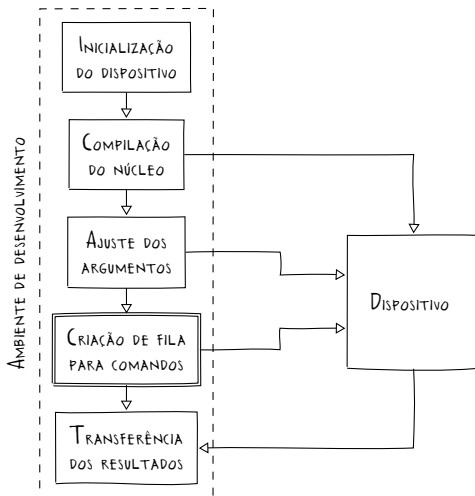
► Fluxo de desenvolvimento



Transferência dos argumentos da função núcleo

Introdução

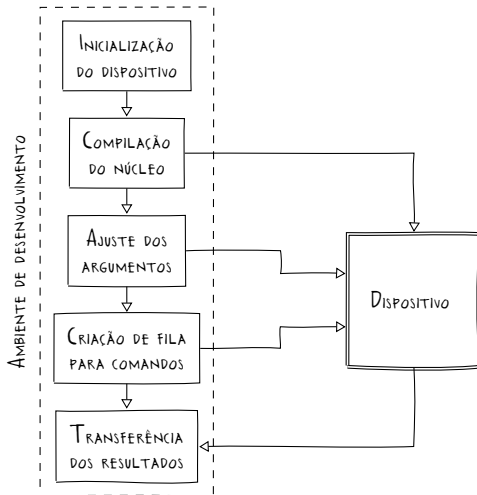
► Fluxo de desenvolvimento



Preparação de fila de comandos para execução

Introdução

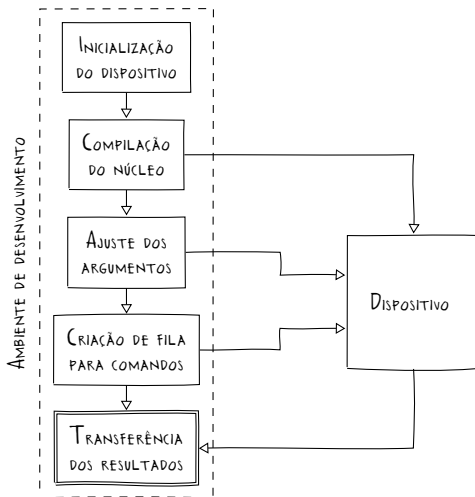
► Fluxo de desenvolvimento



Processamento dos comandos da fila

Introdução

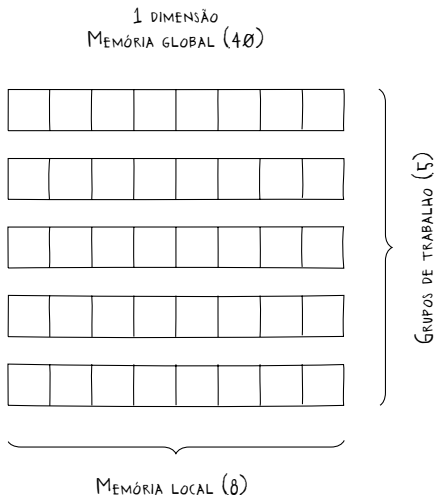
► Fluxo de desenvolvimento



Transferência dos resultados gerados pelo dispositivo

OpenCL

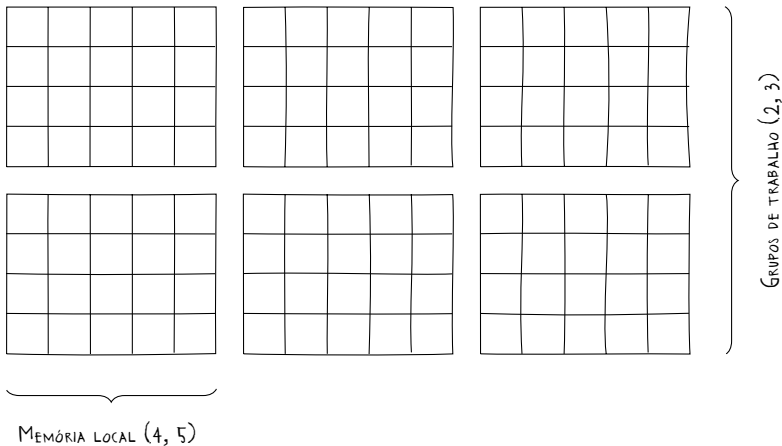
- ▶ Modelo de memória
 - ▶ Global $\times$ Local



OpenCL

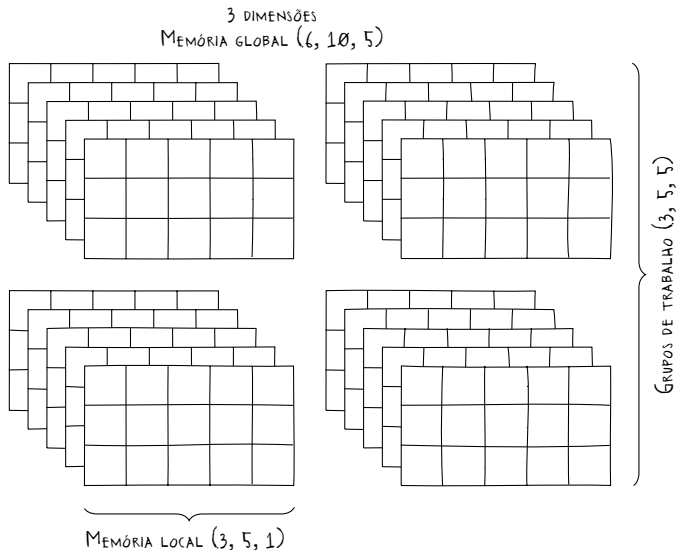
- ▶ Modelo de memória
 - ▶ Global $\times$ Local

2 DIMENSÕES
MEMÓRIA GLOBAL (8, 15)



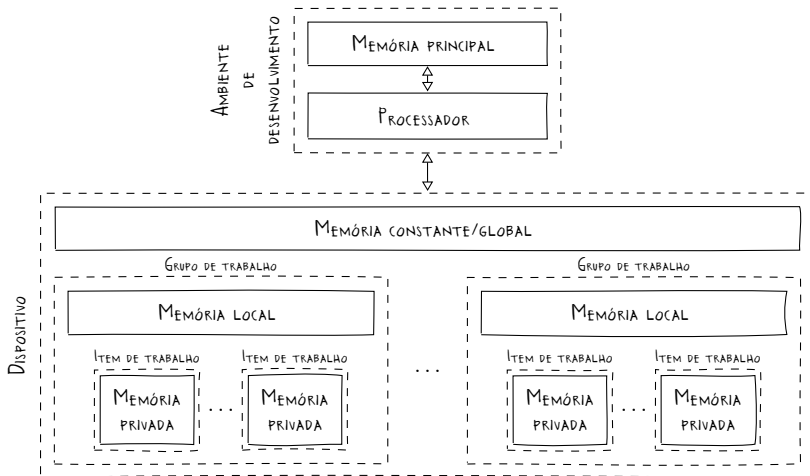
OpenCL

- ▶ Modelo de memória
 - ▶ Global $\times$ Local



OpenCL

- ▶ Modelo de memória
 - ▶ Global × Local × Privada



► Principais funções para acesso dos itens de trabalho

```
1 // Número de dimensões em uso
2 uint get_work_dim();
3 // Número de itens de trabalho globais para dimensão
4 size_t get_global_size(uint dimindx);
5 // Item de trabalho global para dimensão
6 size_t get_global_id(uint dimindx);
7 // Número de itens de trabalho locais para dimensão
8 size_t get_local_size(uint dimindx);
9 // Item de trabalho local para dimensão
10 size_t get_local_id(uint dimindx);
11 // Número de grupos de trabalho na dimensão
12 size_t get_num_groups(uint dimindx);
13 // Grupo de trabalho para dimensão
14 size_t get_group_id(uint dimindx);
```

OpenCL

- ▶ Núcleo para adição vetorial
 - ▶ Ao invés de operar iterativamente com os dados escalares, como geralmente é feito em processadores, as operações vetoriais são executadas paralelamente pelo hardware

```
1  /**
2   *  Adição vetorial com OpenCL
3   *  @param a Entrada a
4   *  @param b Entrada b
5   *  @param c Saída c
6   */
7  __kernel void add_opengl(__global int* a, __global int*
   b, __global int* c) {
8      // Índice do vetor
9      int i = get_global_id(0);
10     // c = a + b
11     c[i] = a[i] + b[i];
12 }
```

► Definições de bibliotecas

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
3 // Definições de OpenCL
4 #include <CL/opencl.hpp>
5 // Asserções em C++
6 #include <cassert>
7 // Tempo em C++
8 #include <chrono>
9 // E/S em terminal
10 #include <iostream>
11 // E/S em arquivo
12 #include <fstream>
13 // Container de vetor
14 #include <vector>
15 // Importando pacotes
16 using namespace std;
17 using namespace std::chrono;
18 ...
```


OpenCL

► Instanciação de componentes

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
3 ...
18 // Marcadores de tempo de alta resolução
19 microseconds cpu, opencl;
20 // Dispositivo da plataforma
21 cl::Device device;
22 // Recursos do dispositivo
23 cl::Context context;
24 // Programa do dispositivo
25 cl::Program program;
26 // Tamanho dos vetores (64 MiB)
27 constexpr uint32_t N = (64 << 20) >> 2;
28 // Vetores no ambiente de desenvolvimento
29 vector<int32_t> a(N, 5);
30 vector<int32_t> b(N, 8);
31 vector<int32_t> c_cpu(N, 1);
32 vector<int32_t> c_opencl(N, 2);
33 ...
```

► Inicialização do dispositivo da plataforma

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
33 // Inicialização do dispositivo
34 void init_device() {
35     // Plataformas OpenCL
36     vector<cl::Platform> platforms;
37     // Dispositivos da plataforma
38     vector<cl::Device> devices;
39     // Obtendo plataformas disponíveis
40     cl::Platform::get(&platforms);
41     ...
42     // Obtendo os dispositivos da plataforma
43     platforms[pindex].getDevices(CL_DEVICE_TYPE_ALL,
44                                  &devices);
45     ...
46     // Setando dispositivo
47     device = devices[dindex];
48 }
```

OpenCL

► Compilação do núcleo

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
89 // Compilação da função do núcleo
90 void compile_kernel(string name) {
91     // Abertura e leitura do arquivo do núcleo
92     ifstream file(name);
93     string kernel(istreambuf_iterator<char>(file),
94                   (istreambuf_iterator<char>()));
95     // Obtendo código fonte do núcleo
96     cl::Program::Sources source({ kernel });
97     // Obtendo recursos do dispositivo
98     context = cl::Context(device);
99     // Setando programa do dispositivo
100     program = cl::Program(context, source);
101     // Compilando programa para o dispositivo
102     cl_int status = program.build();
103     ...
106 }
```

► Adição de vetores com OpenCL

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
...
115 // Adição de vetores (OpenCL)
116 void add_opengl(int32_t* a, int32_t* b, int32_t* c) {
...
    ...
119 // Compilando núcleo
120 compile_kernel("add.cl");
...
    ...
126 // Alocando a memória no dispositivo
127 cl::Buffer buffer_a(context, CL_MEM_READ_ONLY |
    CL_MEM_HOST_NO_ACCESS | CL_MEM_COPY_HOST_PTR, N
    * sizeof(int32_t), a);
128 cl::Buffer buffer_b(context, CL_MEM_READ_ONLY |
    CL_MEM_HOST_NO_ACCESS | CL_MEM_COPY_HOST_PTR, N
    * sizeof(int32_t), b);
129 cl::Buffer buffer_c(context, CL_MEM_WRITE_ONLY |
    CL_MEM_HOST_READ_ONLY, N * sizeof(int32_t));
...
    ...
161 }
```

► Adição de vetores com OpenCL

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
...
115 // Adição de vetores (OpenCL)
116 void add_opengl(int32_t* a, int32_t* b, int32_t* c) {
...
...
135 // Ajustando parâmetros do núcleo
136 cl::Kernel kernel(program, "add_opengl");
137 kernel.setArg(0, buffer_a);
138 kernel.setArg(1, buffer_b);
139 kernel.setArg(2, buffer_c);
...
...
142 // Criando fila de comandos do dispositivo
143 cl::CommandQueue queue = cl::CommandQueue(context,
        device);
...
...
161 }
```

► Adição de vetores com OpenCL

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
...
115 // Adição de vetores (OpenCL)
116 void add_opengl(int32_t* a, int32_t* b, int32_t* c) {
...
...
148 // Enfileirando comando com dimensão global N
149 queue.enqueueNDRangeKernel(kernel, cl::NullRange,
    cl::NDRange(N));
...
...
154 // Leitura dos resultados do dispositivo para o
    ambiente de desenvolvimento
155 queue.enqueueReadBuffer(buffer_c, CL_TRUE, 0, N *
    sizeof(int32_t), c);
...
...
159 // Finalizando a fila de comandos
160 queue.finish();
161 }
```

► Execução comparativa (CPU × OpenCL)

```
$ g++ -Wall -O3 -std=c++0x -o add.elf add.cpp -lOpenCL
$ ./add.elf
...
Global memory size:      2099904512
Local memory size:      49152
Max compute units:      5
Max work item size:     [1024, 1024, 64]
Max work group size:    1024
Choose your device [0 - 0]: 0
-----

Kernel compilation:      57897 us
Buffer allocation:      50024 us (2683.07 MiB/s)
Queue instantiation:     9 us
Kernel execution:       46588 us
Buffer read:            24717 us (2715.09 MiB/s)
-----

Performance report
-----

CPU:                     12852 us
OpenCL:                  185927 us
CPU x OpenCL:            14.4668x
OpenCL x CPU:            0.0691239x
$
```

- ▶ Considerações sobre o desempenho no dispositivo
 - ▶ Grande parte do tempo de execução da adição vetorial em OpenCL foi consumido na compilação do núcleo e na preparação e transferência dos dados entre o ambiente de desenvolvimento e o dispositivo

- ▶ Considerações sobre o desempenho no dispositivo
 - ▶ Grande parte do tempo de execução da adição vetorial em OpenCL foi consumido na compilação do núcleo e na preparação e transferência dos dados entre o ambiente de desenvolvimento e o dispositivo
 - ▶ A complexidade da operação é $O(n)$, onde n é o tamanho dos vetores e cada operação de adição é realizada em memória global com custo $O(1)$

- ▶ Considerações sobre o desempenho no dispositivo
 - ▶ Grande parte do tempo de execução da adição vetorial em OpenCL foi consumido na compilação do núcleo e na preparação e transferência dos dados entre o ambiente de desenvolvimento e o dispositivo
 - ▶ A complexidade da operação é $O(n)$, onde n é o tamanho dos vetores e cada operação de adição é realizada em memória global com custo $O(1)$
 - ▶ Além de serem paralelizáveis, é preciso que as operações realizadas no dispositivo apresentem uma complexidade computacional que compense todo o custo associado à utilização do dispositivo

OpenCL

► Núcleo para multiplicação de matrizes

```
1  /**
2   *  Multiplicação de matrizes com OpenCL
3   *  @param A Matriz A
4   *  @param B Matriz B
5   *  @param C Matriz C
6   *  @param n No. de linhas de A
7   *  @param k No. de colunas de A e de linhas de B
8   *  @param m No. de colunas de B
9   */
10 __kernel void mulmat1_opencl(__global int* A, __global
    int* B, __global int* C, const unsigned int n, const
    unsigned int k, const unsigned int m) {
11     // Índices da linha e coluna
12     unsigned int i = get_global_id(0);
13     unsigned int j = get_global_id(1);
14     ...
23 }
```

OpenCL

► Núcleo para multiplicação de matrizes

```
1  /**
2   *  Multiplicação de matrizes com OpenCL
3   *  @param A Matriz A
4   *  @param B Matriz B
5   *  @param C Matriz C
6   *  @param n No. de linhas de A
7   *  @param k No. de colunas de A e de linhas de B
8   *  @param m No. de colunas de B
9   */
10 ...
11     ...
12     int sum = 0;
13     // Iterando sobre linhas de A e colunas de B
14     for(unsigned int x = 0; x < k; x++) {
15         // Calculando produto linha por coluna
16         sum = sum + (A[i * k + x] * B[x * m + j]);
17     }
18     // Atribuindo resultado em C
19     C[i * m + j] = sum;
20 }
21
22
23 }
```

OpenCL

► Multiplicação de matrizes com OpenCL

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
127 // Multiplicação de matrizes (OpenCL)
128 void mulmat1_opencl(int32_t* A, int32_t* B, int32_t* C,
    const uint32_t n, const uint32_t k, const uint32_t
    m) {
...
    ...
138 // Alocando a memória no dispositivo
139 cl::Buffer buffer_A(context, CL_MEM_READ_ONLY |
    CL_MEM_HOST_NO_ACCESS | CL_MEM_COPY_HOST_PTR, N
    * K * sizeof(int32_t), A);
140 cl::Buffer buffer_B(context, CL_MEM_READ_ONLY |
    CL_MEM_HOST_NO_ACCESS | CL_MEM_COPY_HOST_PTR, K
    * M * sizeof(int32_t), B);
141 cl::Buffer buffer_C(context, CL_MEM_WRITE_ONLY |
    CL_MEM_HOST_READ_ONLY, N * M * sizeof(int32_t));
...
    ...
176 }
```

OpenCL

► Multiplicação de matrizes com OpenCL

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
...
127 // Multiplicação de matrizes (OpenCL)
128 void mulmat1_opengl(int32_t* A, int32_t* B, int32_t* C,
    const uint32_t n, const uint32_t k, const uint32_t
    m) {
...
    ...
163 // Enfileirando comando com dimensão global N x M
164 queue.enqueueNDRangeKernel(kernel, cl::NullRange,
    cl::NDRange(N, M));
...
    ...
169 // Leitura dos resultados do dispositivo para o
    ambiente de desenvolvimento
170 queue.enqueueReadBuffer(buffer_C, CL_TRUE, 0, N * M
    * sizeof(int32_t), C);
...
    ...
174 // Finalizando a fila de comandos
175 queue.finish();
176 }
```

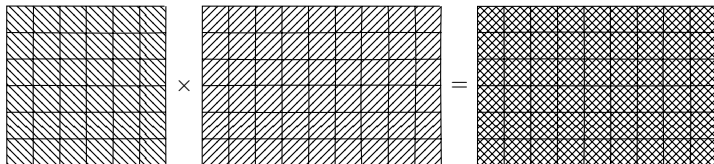
► Execução comparativa (CPU × OpenCL)

```
$ g++ -Wall -O3 -std=c++0x -o mulmat1.elf mulmat1.cpp -lOpenCL
$ ./mulmat1.elf
...
Global memory size:      2099904512
Local memory size:      49152
Max compute units:      5
Max work item size:     [1024, 1024, 64]
Max work group size:    1024
Choose your device [0 - 0]: 0
-----
Kernel compilation:      267870 us
Buffer allocation:       13156 us (2550.5 MiB/s)
Queue instantiation:     15 us
Kernel execution:        12688 us
Buffer read:             5498708 us (3.05112 MiB/s)
-----
Performance report
-----
CPU:                      129823108 us
OpenCL:                   5793708 us
CPU × OpenCL:             0.0446277x
OpenCL × CPU:            22.4076x
$
```

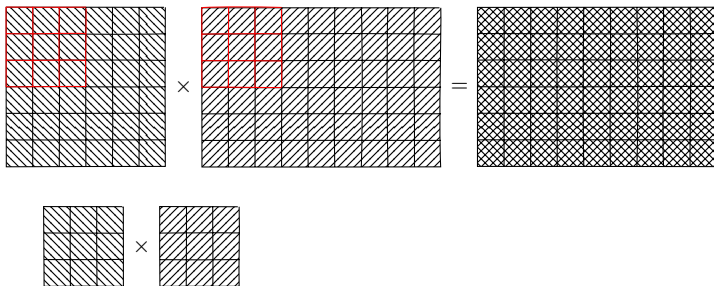
- ▶ Considerações sobre o desempenho no dispositivo
 - ▶ A complexidade de tempo da multiplicação de matrizes $C_{n \times m} = A_{n \times k} \times B_{k \times m}$ é $O(n \times m \times k)$, com todos os acessos e operações realizados na memória global

- ▶ Considerações sobre o desempenho no dispositivo
 - ▶ A complexidade de tempo da multiplicação de matrizes $C_{n \times m} = A_{n \times k} \times B_{k \times m}$ é $O(n \times m \times k)$, com todos os acessos e operações realizados na memória global
 - ▶ O número de acessos à memória global e a latência das operações podem ser reduzidos com a utilização da memória local associada ao grupo de trabalho

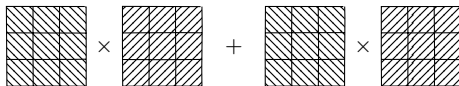
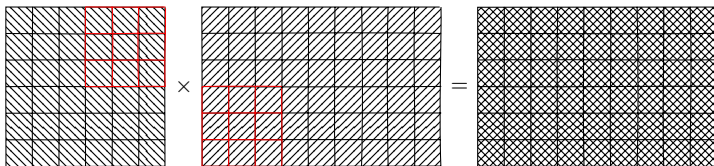
- ▶ Multiplicação de matrizes com OpenCL
 - ▶ Operações em bloco



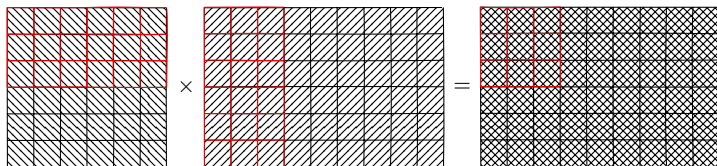
- ▶ Multiplicação de matrizes com OpenCL
 - ▶ Operações em bloco



- ▶ Multiplicação de matrizes com OpenCL
 - ▶ Operações em bloco



- ▶ Multiplicação de matrizes com OpenCL
 - ▶ Operações em bloco



OpenCL

► Núcleo para multiplicação de matrizes

```
1  /**
2   *  Multiplicação de matrizes com OpenCL
3   *  @param A Matriz A
4   *  @param B Matriz B
5   *  @param C Matriz C
6   *  @param n No. de linhas de A
7   *  @param k No. de colunas de A e de linhas de B
8   *  @param m No. de colunas de B
9   */
10 __kernel void mulmat2_opencl(__global int* A, __global
    int* B, __global int* C, const unsigned int n, const
    unsigned int k, const unsigned int m) {
11     // Sub-matrizes locais
12     __local int 1A[MY_LOCAL_SIZE][MY_LOCAL_SIZE];
13     __local int 1B[MY_LOCAL_SIZE][MY_LOCAL_SIZE];
14     ...
36 }
```

► Núcleo para multiplicação de matrizes

```
1  /**
2   *   Multiplicação de matrizes com OpenCL
3   *   ...
9   */
10  ...
14  // Índices da linha e coluna
15  unsigned int i = get_global_id(0), li =
    get_local_id(0);
16  unsigned int j = get_global_id(1), lj =
    get_local_id(1);
17  // Valor da multiplicação
18  int sum = 0;
19  ...
36 }
```

OpenCL

► Núcleo para multiplicação de matrizes

```
1  /**
2   *  Multiplicação de matrizes com OpenCL
3   *  ...
9   */
10  ...
19  // Iterando nas sub-matrizes
20  for(unsigned int x = 0; x < k / MY_LOCAL_SIZE; x++)
21  {
22      // Copiando sub-matrizes para memória local
23      lA[li][lj] = A[(i * k) + ((x * MY_LOCAL_SIZE) +
24                      lj)]];
25      lB[li][lj] = B[j + ((x * MY_LOCAL_SIZE + lj) *
26                      m)];
27      // Sincronização dos itens de trabalho
28      barrier(CLK_LOCAL_MEM_FENCE);
29      ...
36 }
```


OpenCL

► Núcleo para multiplicação de matrizes

```
1  /**
2   *  Multiplicação de matrizes com OpenCL
3   *  ...
9   */
10
11     ...
26     // Iterando sobre linhas de A e colunas de B
27     (local)
28     for(unsigned int y = 0; y < MY_LOCAL_SIZE; y++)
29     {
30         // Calculando produto linha por coluna
31         sum = sum + (1A[li][y] * 1B[y][lj]);
32     }
33     // Sincronização dos itens de trabalho
34     barrier(CLK_LOCAL_MEM_FENCE);
35 }
36 // Atribuindo resultado em C
37 C[i * m + j] = sum;
```

► Multiplicação de matrizes com OpenCL

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
3 ...
93 // Compilação da função do núcleo
94 void compile_kernel(string name) {
95     ...
104     // Compilando programa para o dispositivo
105     cl_int status =
        program.build(string("-DMY_LOCAL_SIZE=" +
            to_string(MY_LOCAL_SIZE)).c_str());
96     ...
110 }
```

► Multiplicação de matrizes com OpenCL

```
1 // Versão do OpenCL
2 #define CL_HPP_TARGET_OPENCL_VERSION 300
...
129 // Multiplicação de matrizes (OpenCL)
130 void mulmat2_opengl(int32_t* A, int32_t* B, int32_t* C,
    const uint32_t n, const uint32_t k, const uint32_t
    m) {
    ...
165 // Enfileirando comando com dimensão global N x M e
    local MY_LOCAL_SIZE x MY_LOCAL_SIZE
166 queue.enqueueNDRangeKernel(kernel, cl::NullRange,
    cl::NDRange(N, M), cl::NDRange(MY_LOCAL_SIZE,
    MY_LOCAL_SIZE));
    ...
178 }
```

► Execução comparativa (CPU × OpenCL)

```
$ g++ -Wall -O3 -std=c++0x -o mulmat2.elf mulmat2.cpp -lOpenCL
$ ./mulmat2.elf
...
Global memory size:      2099904512
Local memory size:      49152
Max compute units:      5
Max work item size:     [1024, 1024, 64]
Max work group size:    1024
Choose your device [0 - 0]: 0
-----
Kernel compilation:      50103 us
Buffer allocation:      10463 us (3206.96 MiB/s)
Queue instantiation:     9 us
Kernel execution:       11733 us
Buffer read:            433594 us (38.6934 MiB/s)
-----
Performance report
-----
CPU:                    131283421 us
OpenCL:                 507134 us
CPU × OpenCL:           0.00386289x
OpenCL × CPU:           258.873x
$
```

- ▶ Documentação e exemplos
 - ▶ <https://www.khronos.org/opencv/>
 - ▶ <https://github.com/KhronosGroup/OpenCL-CLHPP>
 - ▶ <https://github.com/michel-meneses/great-opencv-examples>

Exercício

- ▶ Aprofunde e reproduza os experimentos vistos em OpenCL e faça um comparativo de desempenho da multiplicação de matrizes com memória local versus uma implementação baseada em *multithreading*
 - ▶ Pesquise as principais aplicações que são utilizadas em plataformas de hardware heterogêneas
 - ▶ Verifique o que é necessário para depuração do comportamento do núcleo no dispositivo
 - ▶ Entenda como executar de forma cooperativa e paralela as operações no ambiente de desenvolvimento e nos dispositivos das plataformas